

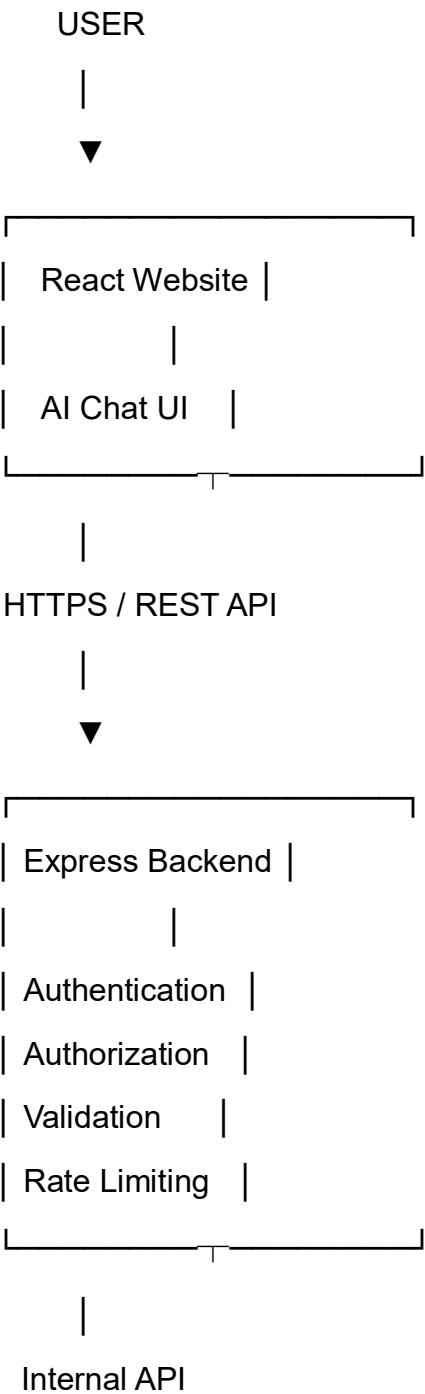
Website Integration — HisabDo AI Copilot

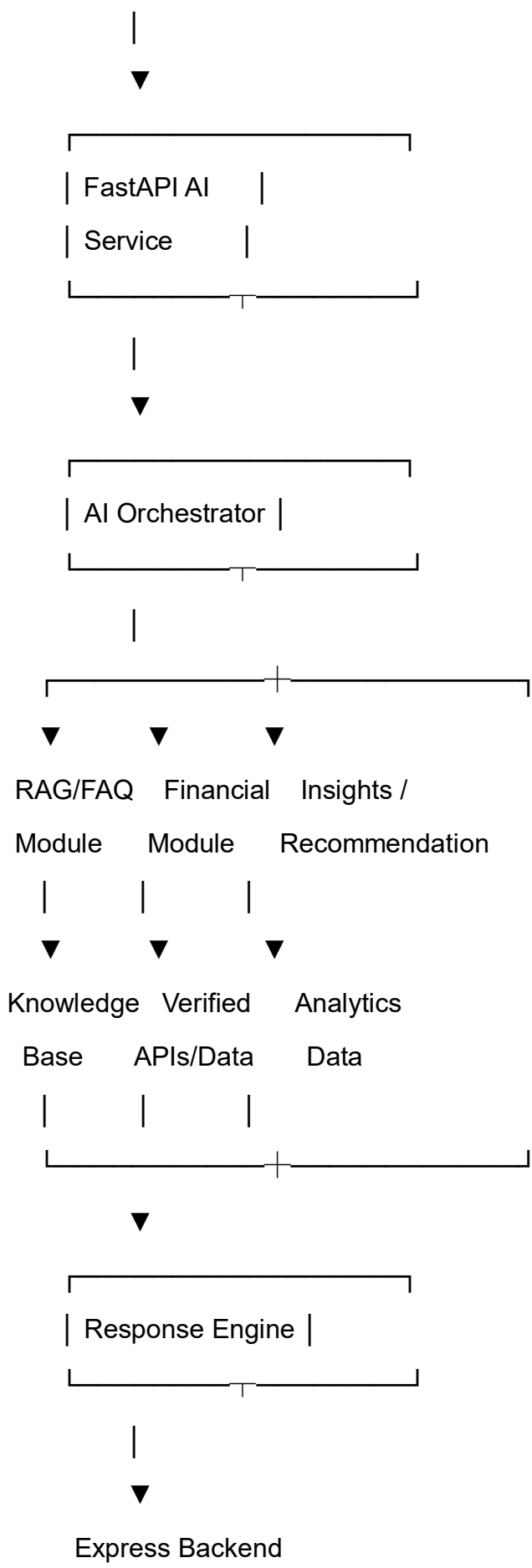
Owner: Muhammad Taha

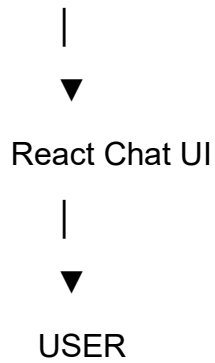
Module: AI/ML — Website Integration

Purpose: Define and validate how HisabDo's existing React website communicates with the Express backend and AI service while keeping financial data secure and user-scoped.

1. Integration Architecture







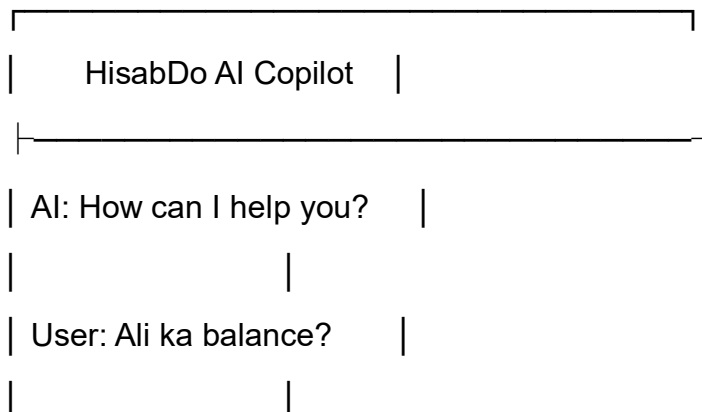
2. React Chat UI

The frontend will provide the user-facing AI interaction.

Requirements

- Chat widget/page
- Message input
- Send button
- Loading state
- AI response display
- Error state
- Conversation history
- Suggested questions
- Multilingual input support
- Mobile responsiveness
- Clear financial response formatting

Example:



AI: Ali ka balance	
PKR 25,000 hai.	

Ask something... [▶]	

3. Frontend → Express Communication

React should **not directly communicate with the database or internal AI modules**.

Flow:

React

↓

POST /api/ai/chat

↓

Express

Example request:

```
{
  "message": "Ali ka kitna udhaar hai?",
  "conversationId": "conv_123",
  "language": "roman-urdu"
}
```

The frontend should send only the information required for processing.

4. Express Backend Responsibilities

Express acts as the main application gateway.

It should handle:

- Authentication
- Authorization

- Request validation
- User identification
- Rate limiting
- Request logging
- AI service communication
- Error handling
- Response formatting

Important:

React

✗ → Database

✗ → FastAPI directly

✗ → LLM directly

React

✓ → Express Backend

5. Express → FastAPI Communication

After authentication and validation, Express forwards the request to the AI service.

React

↓

Express

↓

FastAPI

Example internal request:

POST /ai/chat

Content-Type: application/json

Authorization: Bearer <internal-token>

{

 "userId": "authenticated-user-id",

```
"message": "Ali ka kitna udhaar hai?",  
"conversationId": "conv_123",  
"language": "roman-urdu"  
}
```

The userId must come from the authenticated backend context rather than trusting a user-supplied arbitrary ID.

6. FastAPI AI Service

FastAPI provides the AI-specific layer.

Responsibilities:

- Input processing
 - Language detection
 - Normalization
 - Intent detection
 - AI Orchestrator
 - RAG
 - FAQ processing
 - Financial intent handling
 - Recommendation handling
 - Response generation
 - AI validation
 - Fallback handling
-

7. AI Orchestrator

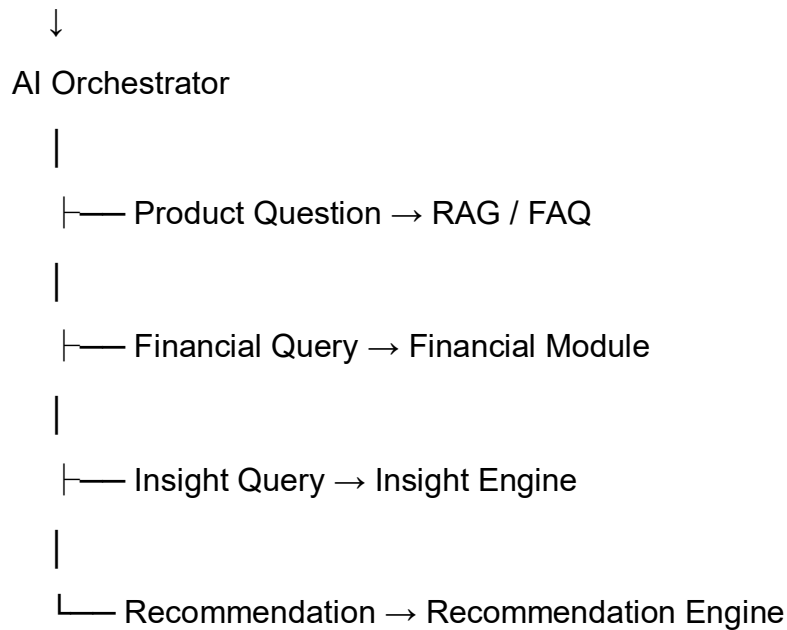
The Orchestrator determines which AI module should handle the request.

Example:

User Query



Intent Detection



8. Financial Data Security

Financial queries require additional protection.

Example:

User A



"Ali ka balance batao"



Authenticated User A



User A's Financial Data

The system must never allow:

User A



User B's Financial Data

Financial data must always be filtered using the authenticated user's identity and authorization rules.

9. No Direct LLM Database Access

The LLM should never receive direct database credentials or unrestricted database access.

Incorrect:

LLM → MongoDB

Correct:

LLM



Controlled Tool/API



Authorization



Business Logic



Verified Data



LLM Response

For example:

"Ali ka balance?"



Financial Intent



Customer Balance API



User Authorization



Verified Balance



AI Explanation

10. RAG / Knowledge Integration

Product-related questions should use the knowledge layer.

User Question



Intent Detection



RAG Module



Vector Search



Relevant Knowledge



Verified Context



LLM



Response

Possible sources:

- FAQs
- Product documentation
- Feature descriptions
- Help articles
- User guides
- Website content

11. Standard Chat Response

Recommended response structure:

```
{  
  "success": true,
```

```
"type": "financial_answer",
"message": "Ali ka current balance PKR 25,000 hai.",
"language": "roman-urdu",
"conversationId": "conv_123"
}

For an FAQ:

{
  "success": true,
  "type": "knowledge_answer",
  "message": "You can create a backup from the Backup section.",
  "language": "english",
  "conversationId": "conv_123"
}
```

12. Complete End-to-End Request Flow

User



React Chat UI



Input Validation



POST /api/ai/chat



Express Backend



Authentication



Authorization



User Context



Request Validation



FastAPI AI Service



Language Detection



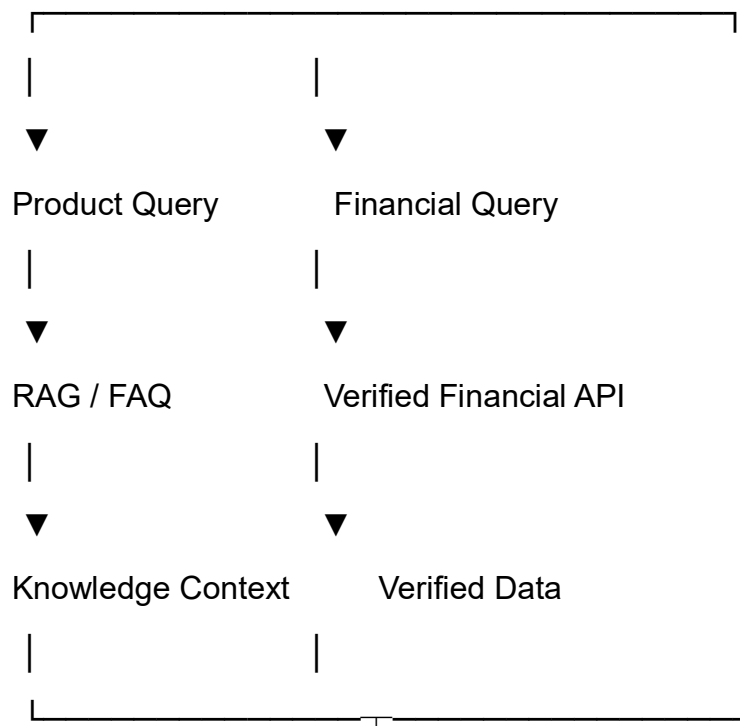
Input Normalization



Intent Detection



AI Orchestrator



Response Generation



Response Validation



FastAPI Response



Express Backend



React Chat UI



User

13. Error Handling & Fallback

The integration should handle:

- AI service unavailable
- Invalid request
- Authentication failure
- Authorization failure
- Timeout
- Empty response
- Knowledge not found
- Financial data unavailable
- Unsupported question

Example:

AI Service



Unavailable



Fallback Handler



"AI Assistant is temporarily unavailable."

Please try again later."

The system should **never fabricate an answer just because a service failed**.

14. Authentication Flow

User Login



Authentication



Access Token / Session



React



Express



Verify User



Attach User Context



AI Service

The AI service should receive trusted user context from the backend.

15. API Endpoints

Public application endpoint

POST /api/ai/chat

Knowledge search

POST /api/ai/knowledge/search

Insights

GET /api/ai/insights

Recommendations

GET /api/ai/recommendations

AI health

GET /api/ai/health

These should be finalized with the MERN team according to the actual HisabDo backend structure.

16. Integration Testing

Test cases should include:

General Query

What is HisabDo?

Expected:

RAG / FAQ

Feature Query

How can I export a PDF?

Expected:

Knowledge Module

Financial Query

Ali ka balance kitna hai?

Expected:

Financial Module

Roman Urdu

Ali ka kitna udhaar hai?

Expected:

Roman Urdu → Financial Intent

Unauthorized Access

Request User B's financial data

Expected:

Access Denied

Unknown Question

Expected:

Safe fallback response

17. Existing Website Protection

AI integration must not break existing HisabDo functionality.

Before integration:

Existing Website



Regression Testing

After integration:

Existing Features



AI Feature



Integration Testing



Regression Testing

Existing modules such as authentication, customers, transactions, reports and other core functionality should continue working independently of the AI service.

18. Dependencies

The integration depends on:

- Existing React application
- Existing Express backend
- Existing authentication system
- Existing user model
- Existing financial APIs
- MongoDB/data layer
- FastAPI AI service

- Knowledge base
 - RAG/vector store
 - LLM service
 - API security configuration
-

19. Implementation Checklist

- ☐ Confirm React integration point
- ☐ Confirm Express API structure
- ☐ Confirm authentication mechanism
- ☐ Define /api/ai/chat contract
- ☐ Define Express → FastAPI contract
- ☐ Define AI response schema
- ☐ Connect AI Orchestrator
- ☐ Connect RAG/FAQ module
- ☐ Define financial API tools
- ☐ Apply user-level authorization
- ☐ Add error handling
- ☐ Add fallback mechanism
- ☐ Test multilingual queries
- ☐ Test financial queries
- ☐ Test unauthorized access
- ☐ Perform regression testing
- ☐ Document final API dependencies

Final Integration Principle

React

↓

Express

↓

FastAPI



AI Orchestrator



Controlled AI Modules



Verified Knowledge / Financial APIs



Validated Response



User

Core rule: The **LLM is an intelligence/explanation layer, not a database or source-of-truth layer**. This keeps the HisabDo AI Copilot safer, more reliable, and easier to integrate with the existing MERN system.